OpenPilot: Phase 1 Conceptual Architecture

EECS 4314 - Advanced Software Engineering
Group A

October 25, 2026

Names	Student Number	Email
Nathan Binu Edappilly	219317965	nathanbe@my.yorku.ca
Bhavya Trivedi	219743590	Bhavya23@my.yorku.ca
Shitij Aggarwal	219299353	billyagg@my.yorku.ca
Roopkiran Kaur	220019204	roopk@my.yorku.ca
Ashdika Siddiqee	219256932	ashdikanusrat@gmail.com

1. Abstract....2
2. Introduction: System Overview and Scope....2
3. Architecture Derivation Method....2
4. System Requirements: Major Use Cases....3
5. Top-Level Conceptual Architecture....4
6. Major Subsystems....5
7. Interactions and Concurrency....13
8. Architectural Styles and Design Patterns....14
9. Alternatives Considered / Derivation Rationale....15
10. Lessons Learned....15
11. Conclusion....15
12. Data Dictionary....16
References....16

1. Abstract

This report presents a conceptual architecture for OpenPilot, an open-source advanced driver-assistance platform. Using official OpenPilot and project documentation, the system's major responsibilities were identified and grouped into conceptual subsystems for perception, localization, planning, control, vehicle adaptation, driver monitoring, and safety. The architecture highlights the major information and control relationships between these subsystems and exhibits both Process-Control and Event-Driven / Publish-Subscribe characteristics. Behavioural diagrams illustrate representative control and engagement flows, while the overall model emphasizes clear responsibilities, continuous feedback, reduced coupling, and safety boundaries without relying on implementation-specific details.

2. Introduction: System Overview and Scope

OpenPilot is an open-source driver-assistance and robotics platform developed by comma.ai. It supports functions such as automated lane centering, adaptive cruise control, lane-change assistance, and driver monitoring. At a high level, the system must interpret the driving environment and vehicle state, determine appropriate driving behaviour, convert those decisions into control actions, and continuously respond to feedback from the physical vehicle.

The purpose of this report is to derive and describe OpenPilot's conceptual architecture. It focuses on major subsystem responsibilities, their interactions, data and control flow, concurrency, and the architectural styles that characterize the system. The report also uses UML behavioural views to illustrate representative system interactions and control-state behaviour.

This phase intentionally abstracts away implementation-specific details such as source files, processes, variables, and low-level communication mechanisms. We derived the conceptual architecture from the official OpenPilot documentation, project documentation, system capabilities, and documented system behaviour to identify the system's major responsibilities and relationships. The resulting architecture is therefore organized around high-level responsibilities such as sensing, perception, localization and state estimation, planning, control, vehicle adaptation, driver monitoring, safety, and supporting services.

3. Architecture Derivation Method

To derive the conceptual architecture, our group followed a structured, multi-step method: First, we thoroughly reviewed official OpenPilot and comma.ai project documentation to understand the platform's overarching capabilities and system requirements. Next, we identified the major operational responsibilities across the system and grouped related responsibilities into distinct conceptual subsystems. We then mapped out the key information and control relationships between these subsystems to define clear data flows and communication boundaries. Finally, we evaluated and compared architectural design alternatives before refining the overall conceptual model into its final structure.

The self-driving system connects speed control (ACC), steering (ALC), and human safety overrides into one cohesive design. Instead of treating these as separate pieces, the system links them together using three simple rules: smart planning, clear safety bounds, and smooth driver handoffs.

- **Data-Driven Planning:** Speed and steering planning use cameras, radar, and predictive computer models instead of rigid rules. The system continuously calculates the safest, smoothest path forward while keeping a comfortable distance from other vehicles.
- **Strict Safety Rules:** AI models can sometimes make unexpected choices. To prevent errors, a simple safety supervisor monitors the AI's planned path. If the driver presses the brake pedal, the supervisor instantly turns off the self-driving features and gives full control back to the driver.
- **Key Design Choices & Trade-offs:**
    - **Fixed Rules vs. Predictive AI:** Basic rule-based systems struggle when lane lines are missing or road conditions change. Using predictive AI models helps the vehicle drive smoothly in real-world traffic.
    - **Full AI Control vs. Step-by-Step Modules:** Letting an AI directly control the steering wheel and pedals without steps in between makes it hard to guarantee safety. Dividing the system into perception, path planning, and steering controls allows engineers to enforce hard safety limits on acceleration and turn angles.

Shared Steering vs. Clear Modes: Trying to blend human steering with AI steering at the same time can cause confusion over who is driving. Using clear, distinct modes ensures the car immediately hands off 100% of control to the driver the moment they turn the wheel or press the pedals.

4. System Requirements: Major Use Cases

At a conceptual level, OpenPilot must satisfy several major system requirements in order to provide driver-assistance functionality safely and continuously.

Requirement	Description
Environment & vehicle-state awareness	Obtain and interpret information about the driving environment, vehicle motion, and current operating state.
Driving decision support	Determine appropriate lateral and longitudinal driving behaviour based on the interpreted environment and vehicle state.
Vehicle support	Translate desired driving behaviour based on the interpreted environment and vehicle state.

Driver monitoring & interaction	Monitor driver attention and allow the driver to override or disengage automated control when required.
Safe vehicle interaction	Ensure that control actions must pass through appropriate safety boundaries before reaching the physical vehicle.
Continuous feedback	Use vehicle-state feedback to adjust subsequent driving and control decisions.

Representative use cases derived from these requirements include:

- Automated Lane Centering: maintain the vehicle near the desired path by continuously adjusting lateral control.
- Adaptive Cruise Control: regulate longitudinal motion to maintain an appropriate speed and following behaviour.
- Lane-Change Assistance: support lateral manoeuvres while retaining driver responsibility for initiating and supervising the manoeuvre.
- Driver Monitoring: monitor driver attention and provide warnings or influence control engagement when necessary.
- Driver Override / Disengagement: allow the driver to immediately interrupt or take priority over automated control.

5. Top-Level Conceptual Architecture

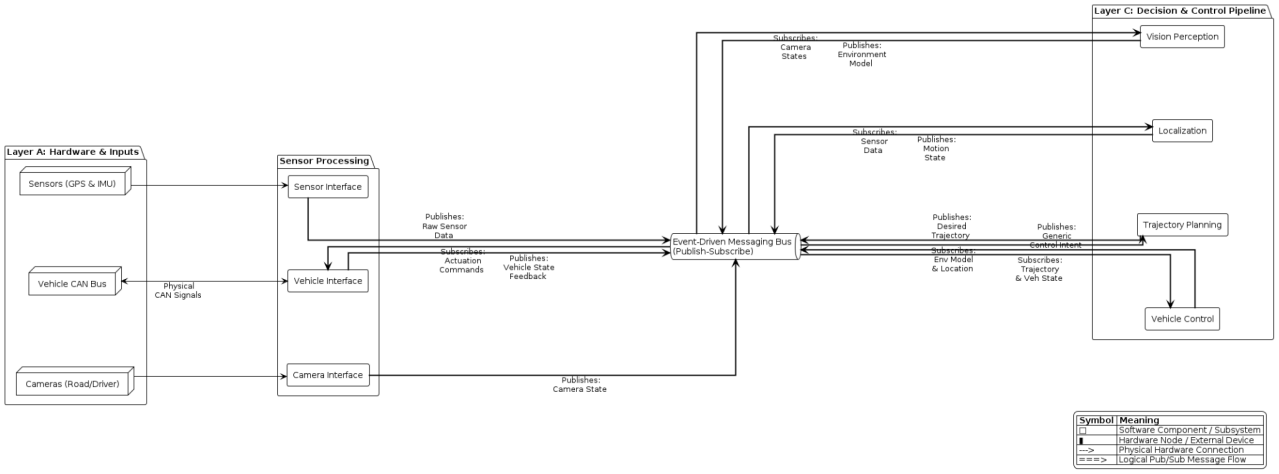


Figure: Top-level architecture diagram

OpenPilot operates as a continuous, closed-loop pipeline where independent daemons collaborate through a centralized messaging bus. The system flow begins with the hardware interfaces, which ingest raw video and sensor data from the physical world and publish it as standardized states to the bus.

The perception and localization subsystems subscribe to this raw data, calculate the vehicle's position, and build a unified environment model (identifying lanes and lead cars), which they publish back to the bus. The planning subsystem then consumes this model to determine the safest path forward, outputting a desired trajectory.

Subsequently, the control subsystem translates that trajectory into a generic control intent, such as specific steering and acceleration commands. Finally, the vehicle interface subscribes to these control intents, translating them into physical actuation commands for the vehicle's CAN hardware. As the car physically moves, the vehicle interface reads the new physical state and publishes it back to the bus, closing the feedback loop for the next cycle. Throughout this entire process, a centralized manager makes sure that all subsystems run concurrently across multiple CPU cores, remaining temporally decoupled by the publish-subscribe architecture.

6. Major Subsystems

6.1 Perception Subsystems

Purpose and Responsibilities

The Perception Subsystem is responsible for interpreting the vehicle's surrounding environment and transforming sensor observations into information that supports autonomous driving decisions. Its primary purpose is to determine what is occurring around the vehicle and provide an accurate representation of the driving environment.

The subsystem analyzes environmental observations to identify relevant road features and traffic conditions. It detects and tracks elements such as lane boundaries, road edges, vehicles, and other objects that may influence vehicle behavior. A key responsibility of the subsystem is estimating road geometry and driving context. This includes determining the vehicle's position relative to roadway features and predicting a feasible future driving path based on current environmental conditions.

The subsystem provides a structured representation of the surrounding environment that can be used by other system components. By isolating environmental interpretation from decision-making and vehicle control, the system supports modularity and independent subsystem evolution.

Inputs: The Perception Subsystem receives:

- Visual observations of the driving environment
- Sensor calibration information
- Vehicle state information
- Configuration and operational status information

Outputs: The Perception Subsystem produces:

- Environmental scene descriptions
- Lane boundary estimates
- Road edge estimates
- Detected and tracked objects
- Predicted driving path information
- Vehicle motion observations derived from environmental sensing

6.2 Localization Subsystem

Purpose and Responsibilities

The Localization Subsystem is responsible for estimating the vehicle's motion, orientation, and dynamic state. Its primary purpose is to determine how the vehicle is moving within its environment. The subsystem integrates information from multiple available sources to maintain a consistent and reliable estimate of vehicle movement. Because individual measurements may be incomplete, noisy, or temporarily unavailable, the subsystem combines complementary information to improve overall accuracy and reliability.

A major responsibility of localization is maintaining an updated estimate of vehicle position, orientation, speed, acceleration, and rotational motion. These estimates provide a common reference that supports planning, decision-making, and control functions. The subsystem also evaluates the reliability of generated estimates and communicates confidence information to downstream subsystems.

Inputs: The Localization Subsystem receives:

- Vehicle-motion observations
- Vehicle-state information
- Sensor calibration information
- Inertial measurements
- Positioning information provided by the overall system architecture

Outputs: The Localization Subsystem produces:

- Vehicle orientation estimates
- Vehicle velocity estimates
- Vehicle acceleration estimates
- Vehicle rotational-motion estimates
- Vehicle-state confidence and validity information

6.3 Interaction Between Perception and Localization

Data Flow and Subsystem Coordination

The Perception and Localization subsystems operate independently while exchanging information through the system communication infrastructure. The Perception Subsystem produces environmental interpretations and motion-related observations derived from sensor data. These observations are provided to the Localization Subsystem as one source of information for estimating vehicle motion. The Localization Subsystem combines these observations with vehicle-state, positioning, and inertial information to generate a consistent estimate of vehicle movement and orientation.

Although the two subsystems collaborate closely, they perform distinct functions. Perception focuses on understanding the external environment, whereas Localization focuses on understanding the vehicle's motion within that environment. Planning and decision-making components use information from both subsystems to determine appropriate driving actions. This separation of responsibilities improves modularity, supports independent testing, and reduces coupling between subsystems.

6.4 Planning / Driving Decision Subsystem

The Planning / Driving Decision subsystem determines the desired vehicle behaviour using environmental information from Perception and vehicle-state estimates from Localization. It determines the intended path, curvature, speed, acceleration, and stopping behaviour before passing these decisions to the Control subsystem.

This separation keeps driving decisions distinct from vehicle actuation. Planning determines what the vehicle should do, while Control determines the lateral and longitudinal actions required to achieve that behaviour.

Inputs: environmental interpretation, vehicle-state estimates, and current driving objectives.

Outputs: desired path or curvature, desired acceleration or deceleration, and stopping or following behaviour.

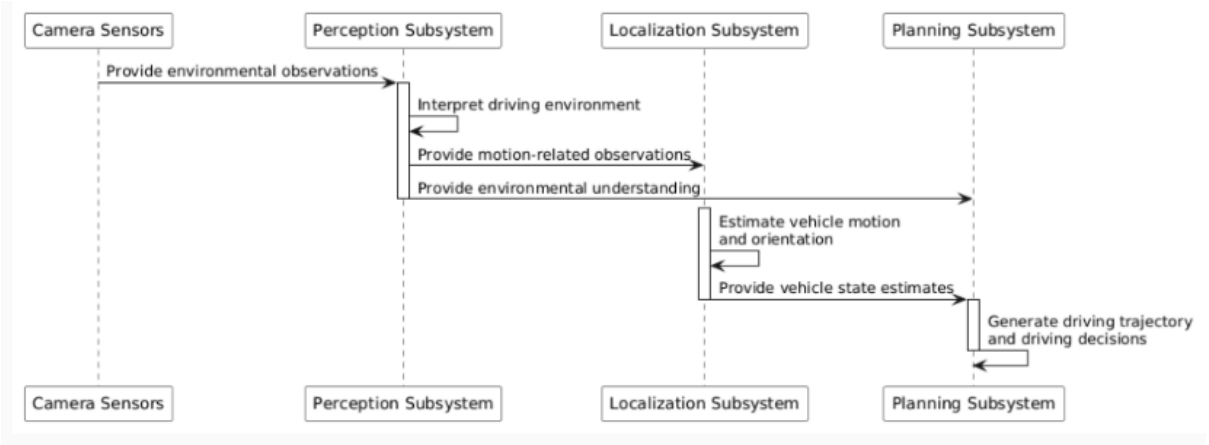


Figure: *Sequence diagram showing how Perception and Localization provide environmental and vehicle-state information to Planning for driving trajectory and decision generation.*

6.5 Control & Vehicle Interface Subsystem

6.5.1 Control Subsystem

The Control subsystem is responsible for converting desired vehicle behaviour produced by upstream driving and planning functionality into control actions that can ultimately be applied to the physical vehicle. Conceptually, it acts as the link between the system's decision-making functions and the Vehicle Interface. Rather than determining where the vehicle should travel, the Control subsystem determines the steering and acceleration behaviour required to achieve the requested motion. [1]

The subsystem can be divided into two primary control responsibilities: lateral control and longitudinal control. Lateral control manages the side-to-side motion of the vehicle by determining the steering action required to follow the desired path or curvature. Longitudinal control manages the forward motion of the vehicle by

determining the acceleration or deceleration required to achieve the desired speed, stopping behaviour, and following behaviour.

Control is not performed as a one-way transformation. The subsystem continuously receives information describing the current state of the vehicle and uses this feedback together with the desired vehicle behaviour to determine appropriate control actions. This feedback relationship allows the controller to respond to changes in vehicle speed, steering state, driver interaction, and other system or vehicle conditions.

Control Inputs & Outputs:

Category	**Conceptual Info**
Vehicle state inputs	Current speed, steering state, and standstill status
Desired behaviour inputs	Desired curvature/path, acceleration, and stopping behaviour
System and safety inputs	Engagement state, driver overrides, faults, and events that may restrict control
Longitudinal Control output	Requested acceleration or deceleration
Lateral Control output	Requested steering or curvature action
Control status output	Whether control functions are active and the current control state

An important architectural boundary exists between the Control subsystem and the Vehicle Interface subsystem. The Control subsystem is concerned primarily with what action the vehicle should perform, such as the required steering or acceleration behaviour. It should remain largely separated from the manufacturer-specific details required to apply that action to a particular vehicle. The Vehicle Interface is responsible for interpreting this higher-level control intent according to the communication and actuation mechanisms supported by the specific vehicle platform.

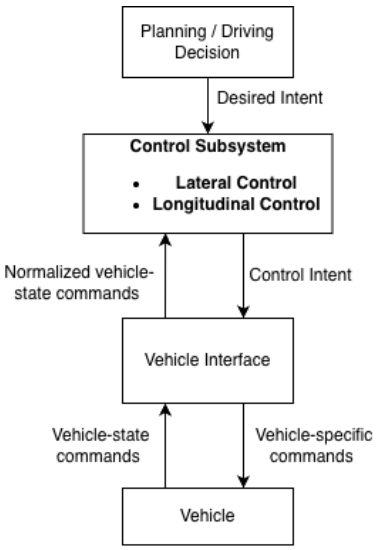


Figure: Conceptual relationship between the Control subsystem, Vehicle Interface, and vehicle-state feedback.

Information about the vehicle's actual state is fed back into the system and used to adjust subsequent control actions. This closed-loop behaviour exhibits characteristics of the Process-Control architectural style, since control decisions are continuously adjusted using feedback about the vehicle's current state [2].

6.5.2 Vehicle Interface Subsystem

The Vehicle Interface subsystem acts as the link between OpenPilot's general control logic and the specific way each supported vehicle communicates. It takes the control requests produced by the Control subsystem and adapts them to the requirements of the detected vehicle. At the same time, it receives information from the vehicle and converts it into a common vehicle-state format that can be understood by Control and other OpenPilot subsystems.

This separation keeps the higher-level control logic from depending too heavily on the communication details of individual vehicles. The Control subsystem can work with a common view of vehicle behaviour and state, while the Vehicle Interface handles the differences between supported platforms. This helps reduce coupling and makes it easier to reuse the same high-level control functions across multiple vehicles.

6.5.3 Panda Safety and Communication Boundary

The Panda safety and communication boundary sits between OpenPilot's Vehicle Interface and the physical vehicle network. Its purpose is not to decide how the vehicle should move, but to provide a controlled path through which vehicle-specific control commands can reach the vehicle and vehicle data can return to OpenPilot. This creates a separation between software-generated control commands and direct access to the vehicle network.

Panda also provides an independent safety boundary around vehicle actuation. Its safety mechanisms can restrict or block vehicle commands when required safety

conditions are not satisfied, rather than relying only on the higher-level control software [3]. This creates an additional layer of protection between OpenPilot's software-generated control requests and the physical vehicle, helping prevent inappropriate or unsafe commands from reaching the vehicle network.

6.5.4 Key Subsystem Interactions and Rationale

The Control and Vehicle Interface subsystems work together as part of a continuous control loop. Each interaction has a specific purpose, allowing OpenPilot to separate high-level driving decisions from vehicle-specific communication while still using feedback from the physical vehicle.

Interaction	Purpose
Planning / Driving Decision → Control	Provides the desired path, curvature, acceleration, and stopping behaviour that Control must achieve.
Control → Vehicle Interface	Passes generic lateral and longitudinal control requests without exposing Control to vehicle-specific communication details.
Vehicle Interface → Panda Safety / Communication Boundary	Sends vehicle-specific control commands through a controlled safety and communication boundary before they reach the physical vehicle.
Vehicle → Panda Safety / Communication Boundary → Vehicle Interface	Returns vehicle-state information through the communication boundary, where it is interpreted and converted into a common representation.
Vehicle Interface → Control	Provides normalized vehicle-state feedback so Control can adjust subsequent steering and longitudinal actions.

Together, these interactions create a closed-loop structure in which desired vehicle behaviour is translated into physical actuation, while the vehicle's actual response is continuously fed back into the system for subsequent control decisions.

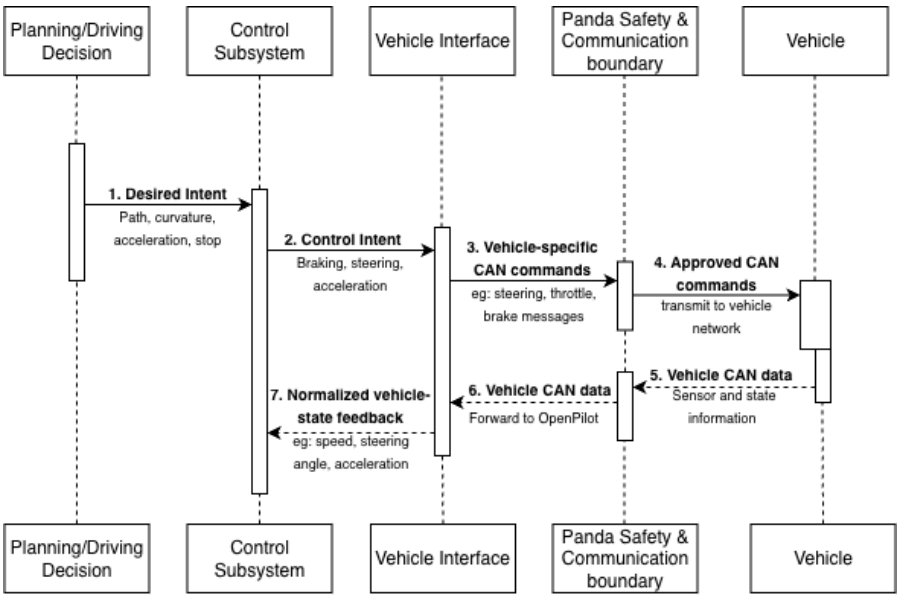


Figure: Interactions of key subsystems

6.5.5 Safety and Fault Handling

OpenPilot applies safety checks at several points before a control request reaches the physical vehicle. At the Control level, actuation may be limited or disabled in response to driver overrides, steering faults, safety-related events, or disengagement conditions. The Vehicle Interface also considers the capabilities and current state of the particular vehicle before control requests are applied.

Panda provides an additional independent safety boundary before commands reach the physical vehicle. It can restrict or block vehicle commands when required safety conditions are not satisfied. Communication failures, unavailable controls, unsafe vehicle conditions, or other detected faults can therefore prevent commands from reaching the vehicle. Driver input can also override or interrupt automated control when required [3].

6.6 Adaptive Cruise Control

Purpose and Responsibilities

The Adaptive Cruise Control (ACC) flow is responsible for regulating vehicle speed and maintaining safe longitudinal distance relative to lead vehicles. Operating as a continuous sense-plan-act loop, raw visual data captured by camerad is processed by modeld to detect lead vehicles and roadway geometry, while locationd integrates deviceMotion and cameraOdometry to track ego-vehicle motion state [1].

The primary responsibility of the longitudinal pipeline is calculating safe acceleration and deceleration profiles based on driver preferences and dynamic lead vehicle behavior. The system continuously evaluates target speeds alongside estimated lead-vehicle distance and relative speed to ensure adequate time-to-collision margins while preserving passenger comfort. These calculated trajectory profiles are subsequently converted by the control and vehicle interface subsystems into platform-specific throttle and braking commands to physically actuate the vehicle [3].

Inputs: The Adaptive Cruise Control flow receives:

- User-configured target speed and following distance preferences from the user interface
- Ego-vehicle speed, acceleration, and dynamic motion state estimates from locationd, deviceMotion, and cameraOdometry
- Lead vehicle distance, relative velocity, and visual tracking outputs generated by modeld from camerad video streams

Outputs: The Adaptive Cruise Control flow produces:

- Safe maximum speed limits and upper trajectory bounds
- Target acceleration and deceleration trajectory profiles

Platform-specific throttle and brake actuation requests

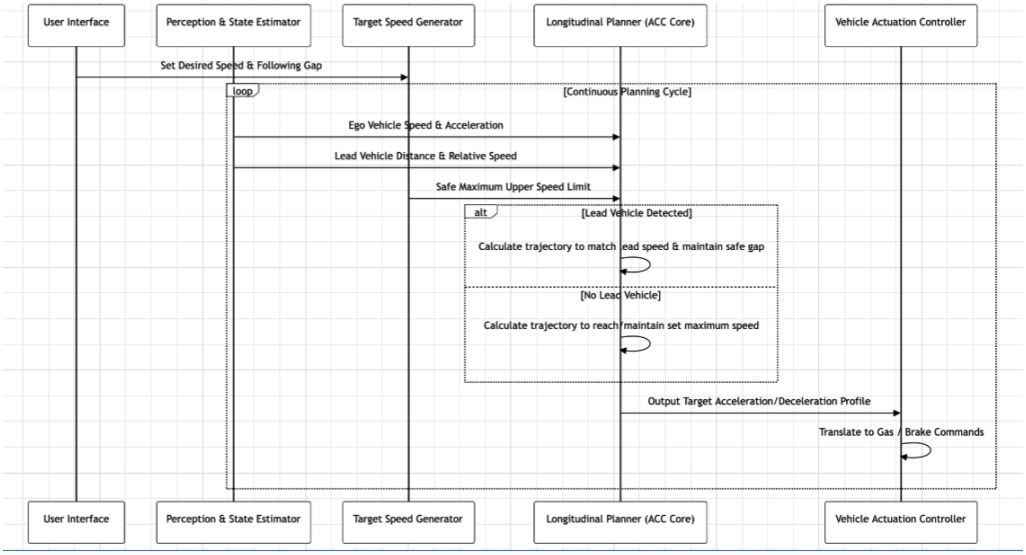


Figure: Automated Cruise Control Sequence Diagram

6.7 Automated Lane Centering

Purpose and Responsibilities

The Automated Lane Centering (ALC) flow is responsible for managing side-to-side vehicle motion to maintain proper positioning within lane boundaries or execute guided lane changes. Video frames ingested by camerad are processed by modeld to generate 3D path predictions and lane line estimates, while locationd provides refined ego-motion state updates combining deviceMotion and cameraOdometry [4].

A core responsibility of the lateral pipeline is evaluating predicted 3D driving paths against current heading, velocity, and roll state to calculate cross-track and heading errors. The system generates target curvature profiles suited for both straight roads and bends, transforming them into steering torque or angle requests for the electronic power steering hardware via the control and vehicle interface layers [1].

Inputs: The Automated Lane Centering flow receives:

- Predicted 3D driving paths and lane boundary estimates output by modeld using camera frames from camerad

- Ego-vehicle heading, orientation, and motion state estimates provided by locationd using deviceMotion and cameraOdometry

Outputs: The Automated Lane Centering flow produces:
- Cross-track and heading error estimates relative to target paths
- Target curvature profiles for straight and curved road segments

Actuation commands for steering torque and angle control

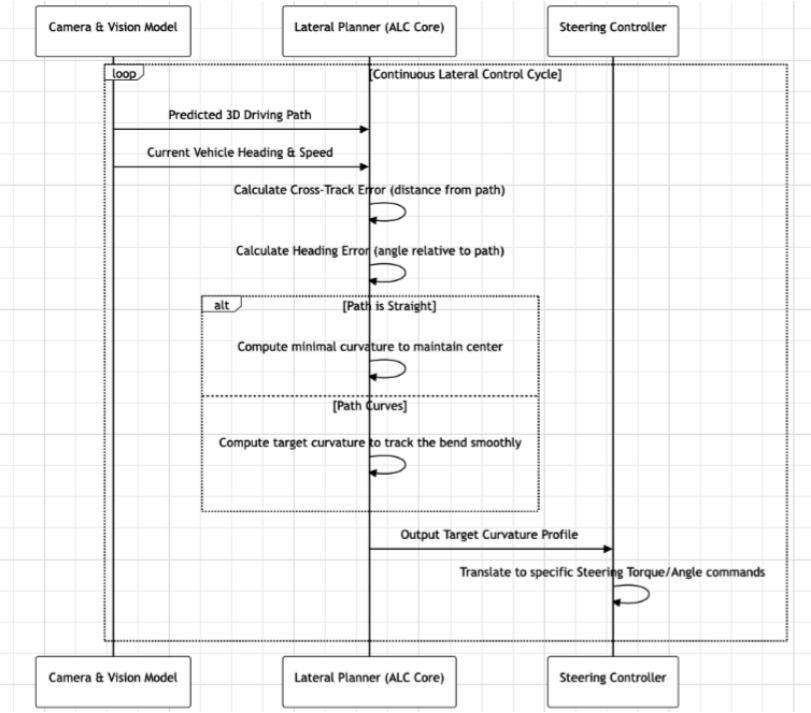


Figure: Automated Lane Control Sequence Diagram

7. Interactions and Concurrency

OpenPilot's major subsystems operate through continuous exchange of information and control. Perception and localization provide information about the driving environment and vehicle state to Planning, which determines the desired driving behaviour. Control translates this behaviour into lateral and longitudinal actions, which are adapted by the Vehicle Interface and passed through the Panda safety and communication boundary before reaching the physical vehicle. Vehicle-state feedback then returns through the architecture and contributes to subsequent control decisions.

These responsibilities operate concurrently rather than as a strictly sequential pipeline. Perception, localization and state estimation, planning, control, vehicle interaction, and driver monitoring continue operating as new information becomes available. This allows driving decisions and control actions to respond continuously to changes in the environment, vehicle state, and driver interaction.

This concurrent exchange of information also contributes to the openpilot's microservice/ Publish-Subscribe characteristics, since cooperating subsystems can

produce and consume updated information without requiring the entire system to execute in a fixed sequence.

8. Architectural Styles and Design Patterns

Openpilot uses a hybrid architecture combining an event-driven publish-subscribe messaging model with microservices. Rather than using a traditional monolithic structure, the system is decomposed into autonomous, asynchronous daemons (such as perception, planning, and control). These microservices do not invoke each other directly; instead, they communicate by publishing and subscribing to standardized data streams on a central messaging bus.

To manage the physical driving task, this event-driven software structure operates alongside a process-control architectural style. The system continuously measures the surrounding environment, calculates a desired driving trajectory, and actuates vehicle controls, forming a closed physical feedback loop where vehicle movement immediately influences subsequent sensor readings. The interaction between event-driven message passing and feedback control is illustrated in the sequence and state diagrams below.

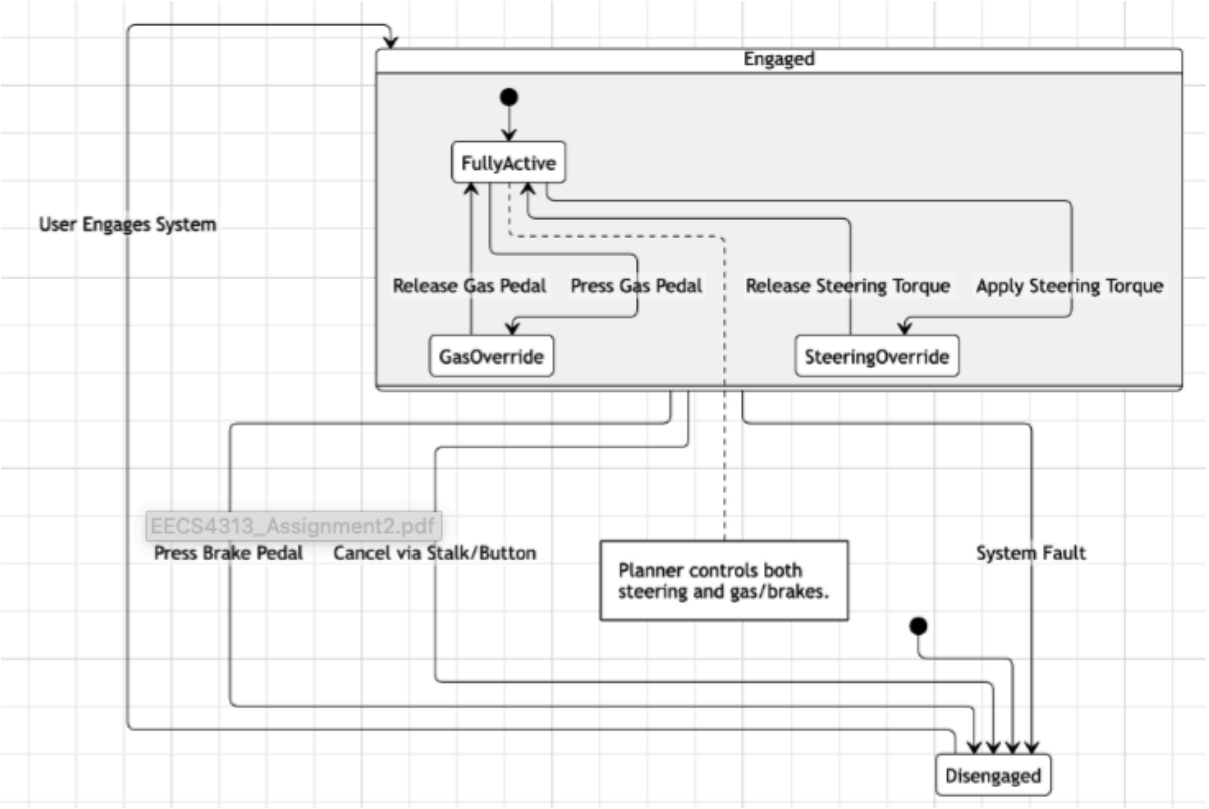


Figure: OpenPilot Control Engagement State Diagram

While data logically flows through Openpilot in a pipeline sequence, from raw sensors to perception, then to planning, and finally to control; it differs from a traditional pipe-and-filter architecture. In a strict pipe-and-filter system, components process data in an open loop and wait synchronously for upstream processing. In contrast, Openpilot relies on a closed-loop feedback cycle and asynchronous execution: microservices independently poll the messaging bus for the latest available state, dropping stale data as needed to maintain real-time performance. [4]

The following diagrams detail these interactions across automated cruise control, automated lane control, and system engagement states.

9. Alternatives Considered / Derivation Rationale

Several design alternatives were evaluated during the conceptual modeling of OpenPilot:

- **Responsibility-Driven Subsystems vs. Repository Mapping:** Decomposing the architecture into logical subsystems based on responsibilities rather than mirroring low-level source files prevents code organization details from obscuring top-level data/control flows and safety boundaries.

- **Multi-Modal Fallback for Driver Monitoring:** Implementing automatic fallback to wheel-touch verification when camera tracking confidence drops ensures continuous monitoring across varied lighting and occlusion conditions.

10. Lessons Learned

1. Keeping parts loosely connected pays off. It lets each part be understood and changed on its own, and it stops a problem in one less-important part from taking down the critical driving parts. The lesson is that in a real-time safety system, that independence is worth the extra indirection.
2. Keep a normal, limited control layer between the learned models and anything that can actually hurt someone. openpilot relies a lot on learned models to understand the road, but it still keeps a control step that enforces limits the models cannot break. The lesson is that learning can decide what you want to do, but the hard safety limits belong in a simple, predictable layer you can check.
3. For anything a human relies on for safety, being trusted matters as much as being accurate. The smoothed warnings exist because a system that warns too often just gets ignored, which is more dangerous than one that is a little slower but trusted. The lesson is that when a person is in the loop, you have to design for false alarms and trust just as carefully as raw accuracy.

11. Conclusion

This report presented a conceptual architecture for OpenPilot based on its major responsibilities and system interactions. The architecture separates sensing, perception, localization, planning, control, vehicle adaptation, driver monitoring, safety, and supporting services while maintaining continuous feedback between the vehicle and the driving-control functions.

Overall, the architecture exhibits both Process-Control and Event-Driven / Publish-Subscribe characteristics, providing a high-level model of how OpenPilot coordinates driving decisions, vehicle actuation, feedback, and safety.

12. Data Dictionary

This section defines the key terms, daemons, messages, and concepts used throughout the OpenPilot conceptual architecture:

- **ACC (Adaptive Cruise Control):** Subsystem responsible for automatically adjusting vehicle speed to maintain a safe distance from preceding traffic.
- **ALC (Automated Lane Centering):** Subsystem responsible for controlling steering actuators to keep the vehicle centered within lane boundaries.
- **camerad:** Hardware interface daemon responsible for capturing raw video frames from road and driver cameras and publishing them to the system message bus.
- **cameraOdometry:** Visual estimation message containing vehicle movement metrics derived from successive camera frame observations.
- **CAN (Controller Area Network):** Vehicle communication bus standard allowing microcontrollers and devices to communicate with each other's applications without a host computer.
- **deviceMotion:** Sensor data stream published by hardware IMUs containing linear acceleration and angular velocity measurements.
- **locationd:** Localization daemon that fuses sensor data (IMU, visual odometry, GPS) to estimate the vehicle's position, orientation, and dynamic motion state.
- **modelId:** Perception vision model daemon that executes neural network inferences on camera frames to output predicted 3D paths, lane line positions, and lead vehicle detections.
- **Panda:** Hardware interface unit and safety layer that provides a secure interface between OpenPilot software and the physical vehicle's CAN bus, enforcing safety limits and command validations.

References

[1] comma.ai, “What is openpilot?,” *openpilot Documentation*, accessed Oct. 23, 2026.
[2] EECS 4314 – Advanced Software Engineering, “Architecture Styles,” York University, 2026, course lecture slides.
[3] comma.ai, “Safety,” *openpilot Documentation*, accessed Oct. 23, 2026.
[4] comma.ai, “A Tour through openpilot,” comma.ai Blog, Dec. 3, 2019. [Online]. Available: https://blog.comma.ai/a-tour-through-openpilot/ . [Accessed: Oct. 23, 2026].